

ABSTRACT

El transporte público continúa siendo uno de los mayores desafíos de movilidad urbana en El Salvador. Aunque miles de personas utilizan autobuses diariamente, no existe una plataforma que permita planificar viajes de forma sencilla, conocer qué rutas tomar, dónde realizar transbordos o cuánto es necesario caminar para llegar al destino.

BusNET propone una solución inteligente inspirada en la experiencia de aplicaciones como Waze o Google Maps, pero enfocada exclusivamente en el transporte público. A partir de la ubicación del usuario y el destino deseado, la aplicación analiza los recorridos de las rutas de autobús para generar una o varias alternativas de viaje, indicando qué buses abordar, dónde realizar transbordos y cuándo es necesario caminar entre rutas cercanas.

Para lograrlo, BusNET combina mapas interactivos, geolocalización en tiempo real, análisis geométrico de recorridos y procesamiento de lenguaje natural, permitiendo que los usuarios puedan buscar destinos utilizando nombres comunes o referencias cotidianas, mientras el sistema interpreta la intención y calcula automáticamente el mejor recorrido.

Nuestro enfoque elimina la dependencia de una base de datos completa de paradas de autobús, modelando directamente los recorridos reales de las rutas y calculando conexiones mediante intersecciones y proximidad geográfica. Esto hace posible desarrollar una solución escalable, adaptable y especialmente útil para ciudades donde la infraestructura del transporte aún no se encuentra completamente digitalizada.

BusNET busca transformar la manera en que las personas utilizan el transporte público, reduciendo la incertidumbre al viajar y ofreciendo una experiencia intuitiva, moderna y accesible para cualquier usuario.

STACK

IDE: Cursor + Cognition (solo revisión)

FrontEnd: React Native + NativeWind + TanStack (Query)

BackEnd: NodeJS + Express

Mapa: MapLibre (motor que dibuja) y ProtoMaps o Carto Basemaps (proporcionan los datos)

Geolocalización: ExpoLocation

Almacenar rutas: GeoJSONs

Calcular la mejor ruta: Turf.js y geometría

Buscar lugar: Nominatim y Ollama (si no se encuentra un lugar, este LLM interpreta el prompt con más detalle y deduce)

Información de las rutas: Firecrawl para scraping.

Voces: ElevenLabs

CREAR TRAYECTOS DE BUSES

Para esto se creará un editor súper sencillo dedicado a conectar puntos y generar los archivos con la ruta de autobús que se pretende representar. Solamente cumplirá las funciones de ver el mapa, tocar un área y colocar un nodo para conectarlo con los demás que se van colocando, eliminar un nodo (se eliminan todos los que van después de este), guardar información en GeoJSONs.

En caso de que el tiempo no sea suficiente, podemos hacer uso de la web [GeoJSON.io](https://geojson.io), que es un editor ya listo para usar, en lugar de reinventar la rueda.

ALMACENAR RUTAS

Las rutas serán almacenadas con la estructura JSON de GeoJSONs, lo cual será crucial para un manejo mucho más cómodo sobre estos datos a la hora de pintar o buscar las mejores rutas para un trayecto al integrarse de buena manera con Turf.js. Solo se almacena el número o nombre de la ruta y los lugares por donde pasa (para pintar su ruta sobre el mapa con MapLibre).

PINTAR MAPA

MapLibre se encarga de hacer el render del mapa en la web, sobre la cual Carto Basemaps coloca el mapa base sobre el que se estará trabajando.

UBICAR LUGARES

Primero se hace la validación con Nominatim y, en caso de no encontrar coincidencias, se hace un llamado al LLM para que interprete la entrada de forma más precisa.

BUSQUEDA DE LA MEJOR RUTA

Buscar la ruta más cercana a la ubicación del usuario en un radio de 100 m aprox. En caso de no encontrar alguna, el radio se expande 100 m más y así sucesivamente.

Calcular y listar las rutas con las que se intercepta el trayecto/ruta de bus más cercana al lugar de destino y a la persona usuaria.

Calcular qué ruta está más cerca del lugar de destino de las rutas que fueron listadas.

Repetir el proceso cuantas veces sea necesario en caso de tomarse más de 2 buses.

(Si no hay un bus que se intercepte con la ruta del bus previo, se busca uno que esté cercano, siguiendo la lógica que se aplica con el radio del principio) ->

Si dos rutas no se intersecan exactamente, calcular el punto más cercano entre ambas para indicar un trayecto caminando.

EXTRAS

- Considerar que, conforme se calculen las rutas, automáticamente se almacenarán los trayectos desde X hasta Y para que así no se tenga que recalcular todo siempre. Se acortan los tiempos de respuesta desde el backend.
- Además, las voces con ElevenLabs serán lo último que se implementará, ya que simplemente es un bomb extra.
- Priorizar primeramente hacer que todo se pueda comunicar entre sí y que funcione. Ya, por último, los estilos.

ETAPAS

1. Renderizar mapa.
2. Tener acceso al GPS.
3. Trazar rutas de autobuses sobre el lienzo.
4. Ubicar la ruta más cercana a la ubicación del usuario.
5. Ingresar, validar y ubicar puntos de destino en el mapa.
6. Buscar rutas cercanas al lugar de destino.
7. Verificar si las rutas se intersecan o si hay que calcular una ruta intermediaria que coincida con ambas o solamente buscar la caminata más cercana, etc.
8. Seleccionar las mejores 4 rutas y listar de más viable a menos recomendable (tráfico, frecuencia de buses activos en esa ruta, tiempo de recorrido, distancia).
9. Pintar dichos trayectos según la opción que se escoja sin considerar el trayecto completo de cada ruta, solamente el trayecto de interés. Lo demás puede aparecer, pero de un color más desvanecido.
10. Automatizar lo anterior para poder ubicar también el punto de partida, ya sea con GPS o escrito.
11. Estilizar todo sin miedo, manteniendo la app intuitiva y limpia. Todo tiene un lugar y un porqué.